SHANGED 9 MONTHS AGO

# 🗓️ Buổi 7: Giới thiệu JavaScript và cú pháp cơ bản

- JavaScript là gì? Vì trí trong web
- Nhúng JS vào HTML (inline, internal, external)
- Biến: var, let, const
- Kiểu dữ liệu: string, number, boolean, null, undefined
- Toán tử: toán học, logic, so sánh
- console.log() và cách debug

## 🟩 1. JavaScript là gì? Vị trí trong web

- JavaScript là gì?

JavaScript (JS) là một ngôn ngữ lập trình dùng để tạo tương tác trên trang web.
Được chạy trên trình duyệt (client-side), nhưng cũng có thể dùng ở server-side (nhu với Node.js).

Là một phần trong bộ ba công nghệ web chính:

- HTML: tạo cấu trúc trang web.
- CSS: tạo kiểu dáng cho trang web.
- JavaScript: thêm hành vi động (nhấp tắc, hiệu ứng, xử lý sự kiện...).

- Đặc điểm nổi bật:

Thông dịch (interpreted): không cần biên dịch trước.
Động (dynamic): kiểu dữ liệu linh hoạt, không cần khai báo cứng.
Hướng đối tượng (object-based): làm việc nhiều với object, hỗ trợ class từ ES6.
Sự kiện (event-driven): ứng ngha và xử lý sự kiện từ người dùng (click, keyup...).

- JavaScript làm được gì?

Hiển thị thông báo, popup, modal
Thay đổi nội dung HTML/CSS động (DOM manipulation)
Kiểm tra form trước khi gửi (form validation)
Tạo slideshow, menu dropdown, animation...
Giao tiếp với server qua AJAX, fetch
Ứng dụng đơn trang (SPA) với React, Vue, Angular

- Vị trí của JavaScript trong website:

Là phần logic điều khiển hành vi.
Được chạy sau khi HTML và CSS được tải.
Có thể tương tác với DOM (Document Object Model) để thay đổi nội dung trang web.

## 🟩 2. Nhúng JavaScript vào HTML: inline, internal, external

- (1) Inline JavaScript

Viết trực tiếp trong thuộc tính HTML, thường dùng với onclick, onchange, onmouseover,...

✅ Ví dụ:

```
<button onclick="alert('Xin chào!')">Click me</button>
```

⚠️ Lưu ý:

Dễ viết, nhưng khó bảo trì, không tách biệt nội dung và logic.
Không nên dùng trong dự án lớn.

- (2) Internal JavaScript

Viết mã JS trực tiếp trong thẻ <script> đặt trong file HTML (thường trong phần <head> hoặc cuối <body>).

✅ Ví dụ:

```
<!DOCTYPE html>
<html>
<head>
  <title>Internal JS</title>
  <script>
    function sayHello() {
      alert("Hello from internal JS!");
    }
  </script>
</head>
<body>
  <button onclick="sayHello()">Click me</button>
</body>
</html>
```

- Vị trí đặt:

Trong <head>: code có thể chạy trước khi DOM sẵn sàng → cần bọc trong window.onload hoặc đặt ở cuối <body>.

Cuối <body>: khuyến nghị hơn → DOM đã tải xong → tránh lỗi truy cập phần tử chưa tồn tại.

- (3) External JavaScript

Viết mã JS trong một file riêng (.js) và gọi bằng thẻ <script src=".">.

✅ Ví dụ:

📄 script.js

```
function sayHello() {
  alert("Hello from external JS!");
}
```

📄 index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>External JS</title>
  <script src="script.js" defer></script>
</head>
<body>
  <button onclick="sayHello()">Click me</button>
</body>
</html>
```

- Ưu điểm:

Tách biệt logic và nội dung → dễ bảo trì, dễ tái sử dụng, tăng tốc độ tải trang (cache)
Tối nhất nên dùng cách này cho dự án thực tế.

| Loại | Ưu điểm | Nhược điểm | Dùng khi nào? |
|---|---|---|---|
| Inline | Dễ viết, truy cập trực tiếp | Khó bảo trì, lẫn logic vào HTML | Thử nghiệm nhỏ, demo đơn giản |
| Internal | Tách logic tốt hơn inline | Vẫn còn trong HTML | Dự án nhỏ, hoặc 1 trang duy nhất |
| External | Rõ ràng, dễ tái sử dụng, hiệu quả | Cần thêm file .js | Dự án thực tế, dễ mở rộng |

## 🟩 3. Biến: var, let, const

- Biến là gì?

Biến (variable) là nơi lưu trữ dữ liệu tạm thời trong bộ nhớ
Có thể khai báo bằng 3 cách: var, let, const

- var – cách cũ (nên tránh dùng)

```
var name = "Quang";
```

Đặc điểm:
Có hoisting: được "đưa lên" đầu phạm vi (function)
Không block scope: không giới hạn trong { }
Có thể gán lại và khai báo lại

- let – hiện đại, nên dùng

```
let age = 25;
```

Đặc điểm:
Có block scope
Có thể gán lại
Không thể khai báo lại trong cùng phạm vi

- const – hằng số

```
const pi = 3.14;
```

Đặc điểm:
Có block scope
Không thể gán lại
Nhưng nếu là object/array → vẫn có thể thay đổi nội dung bên trong

// Hợp đoán kết quả trước khi chạy

```
var x = 1;
{
  var x = 2;
  console.log(x); // ?
}
console.log(x); // ?

let y = 1;
{
  let y = 2;
  console.log(y); // ?
}
console.log(y); // ?
```

## 🟩 4. Kiểu dữ liệu cơ bản trong JavaScript

- JavaScript là ngôn ngữ dynamically typed
Không cần khai báo kiểu dữ liệu rõ ràng
JS sẽ tự hiểu kiểu của biến dựa vào giá trị gán

- Các kiểu dữ liệu nguyên thủy (primitive types):

| Tên | Ví dụ | Ghi chú |
|---|---|---|
| string | "Hello" hoặc 'Xin chào' | Dùng để chứa chuỗi văn bản |
| number | 5 , 3.14 , -100 | Không phân biệt số nguyên và số thực |
| boolean | true , false | Chỉ có 2 giá trị |
| null | null | Đại diện cho giá trị rỗng có chủ ý |
| undefined | undefined | Khi biến được khai báo nhưng chưa gán giá trị |

- Sử dụng typeof để kiểm tra kiểu:

```
console.log(typeof "hello"); // string
console.log(typeof 123);     // number
console.log(typeof true);    // boolean
console.log(typeof null);    // object (lỗi lịch sử của JS)
console.log(typeof undefined); // undefined
```

- Khác biệt giữa null và undefined

| Thuộc tính | null | undefined |
|---|---|---|
| Được gán | Có | Không (mặc định) |
| Ý nghĩa | Không có giá trị | Không xác định |
| typeof | object (bug lịch sử) | undefined |

✅ Bài tập thực hành:

```
// 1. Khai báo các biến tương ứng với các kiểu dữ liệu
let myName = "Lan";
let myAge = 20;
let isStudent = true;
let favorite;
let unknownValue;

// 2. In kiểu dữ liệu từng biến ra console
console.log(typeof myName);     // ?
console.log(typeof myAge);      // ?
console.log(typeof isStudent);  // ?
console.log(typeof favorite);   // ?
console.log(typeof unknownValue); // ?
console.log(typeof undefinedVariable); // ?
```

### 🧠 Ghi nhớ:

Dùng let khi giá trị có thể thay đổi
Dùng const cho giá trị cố định
Tránh dùng var (có thể gây lỗi do hoisting và không có block scope)
JS có 5 kiểu dữ liệu cơ bản: string, number, boolean, null, undefined

## 🟩 5. Toán tử là gì?

Toán tử (operator) là ký hiệu dùng để thực hiện các phép tính hoặc so sánh giữa các giá trị.
Ví dụ: +, -, ==, &&,...

- 🔹 2. Toán tử toán học (Arithmetic operators)

| Toán tử | Ý nghĩa | Ví dụ | Kết quả |
|---|---|---|---|
| + | Cộng | 5 + 2 | 7 |
| - | Trừ | 5 - 2 | 3 |
| * | Nhân | 5 * 2 | 10 |
| / | Chia | 5 / 2 | 2.5 |
| % | Chia lấy dư (modulo) | 5 % 2 | 1 |
| ** | Lũy thừa (ES6) | 2 ** 3 | 8 |
| ++ | Tăng 1 đơn vị | x++ | x = x + 1 |
| -- | Giảm 1 đơn vị | x-- | x = x - 1 |

✅ Bài tập nhỏ:

```
let a = 10;
let b = 3;
console.log(a + b);  // ?
console.log(a % b);  // ?
console.log(a ** b); // ?
console.log(a++, a); // ?
```

- 🔹 3. Toán tử so sánh (Comparison operators)

| Toán tử | Ý nghĩa | Ví dụ | Kết quả |
|---|---|---|---|
| == | Bằng (so sánh giá trị) | 5 == '5' | true |
| === | Bằng (so sánh giá trị và kiểu) | 5 === '5' | false |
| != | Không bằng | 5 != '5' | false |
| !== | Không bằng (giá trị hoặc kiểu) | 5 !== '5' | true |
| > | Lớn hơn | 5 > 3 | true |
| < | Nhỏ hơn | 3 < 5 | true |
| >= | Lớn hơn hoặc bằng | 5 >= 5 | true |
| <= | Nhỏ hơn hoặc bằng | 4 <= 5 | true |

🧠 Lưu ý quan trọng:
== sẽ chuyển đổi kiểu dữ liệu trước khi so sánh
=== là tiêu chuẩn, vì so sánh chặt chẽ (strict comparison)

✅ Bài tập nhỏ:

```
console.log(5 == "5"); // ?
console.log(5 === "5"); // ?
console.log(null == undefined); // ?
console.log(null === undefined); // ?
```

- 🔹 4. Toán tử logic (Logical operators)

Toán tử Tên gọi Mô tả
&& AND Trả về true nếu tất cả vế đều đúng
|| OR (hoặc) Trả về true nếu ít nhất một vế đúng
! NOT (phủ định) Đảo ngược giá trị logic (boolean)

✅ Ví dụ sử dụng:

```
let isAdult = true;
let hasTicket = false;

if (isAdult && hasTicket) {
  console.log("Được vào rạp");
} else {
  console.log("Không đủ điều kiện");
}
```

- 🔹 Truthy & Falsy

Trong JavaScript, không chỉ true và false mới được xem là logic.
Một số giá trị được xem như là false (falsy):

- false, 0, "" (chuỗi rỗng), null, undefined, NaN

```
if ("") console.log("true"); // Không in
if ("hello") console.log("true"); // In ra vì string không rỗng là truthy
```

📌 Bài tập thực hành tổng hợp:

```
// 1. Tính tổng, hiệu, tích, chia của 2 số
let a = 8;
let b = 4;
console.log("Tổng:", a + b);

// 2. So sánh 2 số: a1 lớn hơn không?
if (a > b) {
  console.log("a lớn hơn b");
}

// 3. Kiểm tra điều kiện vào rạp:
let age = 17;
let hasTicket = true;
if (age >= 18 && hasTicket) {
  console.log("Em phải vào rạp");
} else {
  console.log("Không được vào");
}
```

### 🎯 Tổng kết:

- Toán tử toán học giúp thực hiện phép tính
- Toán tử so sánh trả về giá trị true hoặc false
- Toán tử logic dùng để kết hợp nhiều điều kiện
- Luôn ưu tiên dùng === và != để so sánh chính xác

## 🟩 6. Console và cách debug

- 🔹 1. console.log() là gì?

- Định nghĩa:

Là hàm có sẵn trong JavaScript dùng để hiển thị thông tin ra "Console" của trình duyệt hoặc môi trường Node.js.

- Mục đích:

- Kiểm tra giá trị của biến
- Theo dõi dòng thực thi chương trình
- Tìm lỗi trong quá trình chạy

- 🔹 2. Các hàm console thường dùng

| Hàm | Mô tả | Ví dụ |
|---|---|---|
| console.log() | In thông tin chung | console.log("Xin chào"); |
| console.error() | In lỗi (màu đỏ) | console.error("Có lỗi xảy ra!"); |
| console.warn() | In cảnh báo (màu vàng) | console.warn("Cảnh báo: ..."); |
| console.table() | Hiển thị dữ liệu dạng bảng | console.table([{name: "A"}, {name: "B"}]); |
| console.clear() | Xóa toàn bộ console | console.clear(); |

- 🔹 3. Cách dùng DevTools để debug

🧭 Mở Console:
Nhấn F12 hoặc Chuột phải → Inspect → Tab Console
Dùng để xem:

- Kết quả console.log()
- Lỗi cú pháp
- Thông báo từ trình duyệt

🧠 Dùng Debugger:
Giữ breakpoint để dừng chương trình tại một dòng